SQUAD: Scheduling Queue Unblocking and Allocation Defragmentation for Gang Scheduling

Kyunam Cho[1,2,3], YoungHwan Jin[2], Heonchang Yu[1*]

[1*]Department of Computer Science and Engineering, Korea University, Seoul, Republic of Korea.
[2]CTO, LG Uplus Corp., Seoul, Republic of Korea.
[3]School of Business, Ehwa Woman University, Seoul, Republic of Korea.

*Corresponding author(s). E-mail(s): yuhc@korea.ac.kr;
Contributing authors: mystous@korea.ac.kr; yhjin@korea.ac.kr;

Abstract

As a representative example of hardware accelerators or coprocessors, Datacenter GPU is an essential hardware resource for AI research that provides significant computational power albeit at a considerable financial cost. This importance has grown with the widespread adoption of transformer-based large language models (LLMs), which often require dozens or even hundreds of NVIDIA datacenter GPUs for training. Consequently, the efficient allocation and management of GPU resources are crucial. The scheduling problem is a classic NP hard combinatorial optimization problem; mapping jobs with unknown runtimes to a finite set of GPUs cannot be solved optimally in polynomial time. Existing schedulers typically optimize a single metric (e.g., throughput, fairness, or deadline compliance) under specific assumptions. Instead of proposing another scheduler, Scheduling Queue Unblocking and Allocation Defragmentation for Gang Scheduling (SQUAD) introduces a complementary mechanism composed of two algorithms that enhance a broad class of baseline schedulers that do not rely on cumulative or historical state metrics. First, the starvation free queue-adjustment algorithm monitors wait times and resource demands to detect head of line blocking, and reorders the queue to prevent starvation. Second, the preemptive task reallocation algorithm selectively preempts and migrates running jobs to consolidate scattered GPUs when the fragmentation exceeds a threshold. These two mechanisms act as preprocessors that optimize the job queue and server state, enabling any scheduler to operate in a highly efficient resource pool. These interventions are triggered only when the heuristic conditions indicate that the expected reduction in the wait and execution times outweighs the migration overhead. Across multiple workloads and baseline scheduling policies, such as

packing, round-robin, and Monte Carlo tree search, the two SQUAD algorithms reduce the wall clock time by approximately 7.8–14% and increase the GPU allocation rate by approximately 14.98 percentage points. As SQUAD operates at the base regardless of the scheduler layer and leaves the core policies intact, it can be integrated into existing GPU cluster managers to improve its utilization without sacrificing stability.

Keywords: Head-of-Line blocking, reallocation, reorder wait queue, GPU, fragmentation, job queue, waiting queue, Starvation-Free, defragmentation

1 Introduction

Scheduling training or inference jobs on GPU servers is a crucial aspect of system-level optimization. The GPU allocation rate and utilization on the server are critical operational metrics that affect operational and research costs. Moreover, GPU resources are inherently limited to research and business groups, prompting extensive studies for optimal job scheduling. Some organizations form "hyperclusters" which are dedicated clusters of hundreds or thousands of GPUs connected by high-bandwidth interconnects. This approach not only drives up costs but also imposes scalability limits and causes resource fragmentation across clusters[1]. Similarly, an analysis of production traces showed that allocating fractional GPUs in shared environments leads to severe fragmentation, leaving hundreds of GPUs idle and unavailable for incoming jobs[2]. These findings underscore the importance of designing schedulers and defragmentation mechanisms that can effectively manage GPU resources despite increasing model sizes and diverse workloads. However, there is no universal solution because most studies are based on specific conditions or highly idealized scenarios. This is because the problem of assigning tasks with unknown completion times to a queue is a well-known NP-hard combinatorial optimization problem[3]. Most job-scheduling problems involve a combination of flow shops, job shops, single machines, open shops, and k-machine parallel scheduling[4]. Thus, most job-scheduling problems are handled by an approximation algorithm[5].

Nevertheless, research on schedulers for deep learning model training and inference jobs continues. Ye et al.[6] provided a taxonomy of deep-learning job scheduling: first, by workload type (training, inference, etc.), and second, by the specific characteristics and objectives (efficiency, fairness, deadlines, etc.)of each workload. They identified state-of-the-art schedulers, such as Sia[7], which excels in utilization, fairness, and job completion time, and Lucid[8], which uses multi-state profiling to optimize performance. Most existing GPU schedulers adopt variations of these methods. Therefore, if the job characteristics change, scheduling policies or schedulers should be adjusted accordingly.

Therefore, the recently proposed GPU scheduling approaches exhibit inherent limitations. For instance, in a learning-to-rank-based LLM scheduler, Kendall's Tau ranking metric has occasionally been shown to fail to reflect end-to-end performance; random shuffling within the shortest 70% of requests maintains the Tau score (0.5), but

significantly increases latency (1.8× increase observed with Llama-8B on ShareGPT). Furthermore, this method has not yet been fully tested with the integration of newer LLM service optimizations, such as chunk-prefill and prefill-decode disaggregation[9]. Similarly, fragmentation-aware scheduling approaches, such as Fragmentation Gradient Descent (FGD)[10], designed to manage GPU-sharing workloads, focus on a narrow 'one step' measurement of fragmentation to ensure scheduler independence. This narrow focus can lead to suboptimal performance during the initial phases when the clusters are resource-rich and fragmentation is naturally low.

Therefore, these sophisticated scheduling approaches, which are designed to optimize resource allocation, introduce their own complexities and limitations. For instance, dynamic schedulers such as Optimus[11] rely on online resource performance models to estimate training speed and convergence. Although effective, the performance of Optimus is sensitive to prediction errors, particularly when estimating model convergence, where prediction errors of up to 20% have been observed. Furthermore, its reliance on a checkpoint-based method to adjust the resource configuration introduces overhead, which can be significant for jobs involving hundreds of workers or parameter servers. Separately, approaches that target fairness and placement sensitivity, such as Themis[11], achieve their goals by introducing significant architectural and computational complexities. Themis used a centralized round-by-round auction mechanism to determine the resource allocation based on app-provided bids. This mechanism incurs a high computational overhead, reaching 1,398 ms at the 95th percentile during high contention, owing to the use of a solver (e.g., Gurobi) for the partial allocation auction. Moreover, Themis's effectiveness relies on complex inputs from applications to estimate a placement-sensitive finish-time fairness metric, which requires applications to precisely model the slowdown caused by non-ideal GPU placement.

In contrast to these highly complex and overhead-prone approaches, SQUAD focuses on a complementary mechanism that enhances existing scheduling by addressing the core limitations of GPU cluster management. SQUAD does not attempt to solve the NP-hard problem directly; rather, it resolves two major bottlenecks—Head-of-Line (HOL) blocking and resource fragmentation—which are independent of the core scheduling policy and degrade the performance of any compatible scheduler. By addressing these prescheduling inefficiencies, SQUAD ensures that the scheduler operates in an optimized state.

Our approach achieved a 7.8%–14% reduction in job completion time (depending on the workload and baseline scheduler) and improved the GPU allocation rate by approximately 14.98 percentage points. We validated our approach using augmented job logs generated from real GPU server pool data. SQUAD achieves this using two distinct and equally important algorithms.

The key contributions of this study are as follows.

1. **Starvation-Free Queue Adjustment:** When a job requests the maximum number of GPUs on a server, it may experience long wait times if the server has insufficient resources, resulting in starvation. This condition, HOL blocking, delays subsequent jobs even when servers with adequate GPUs are available. Our Starvation-Free Queue Adjustment mechanism specifically monitors the wait times

and resource requirements of queued jobs, dynamically reordering the queue to eliminate HOL blocking and ensure fairness across varying job sizes.
2. **Preemptive Allocation Defragmentation:** Simultaneously, GPU server pools frequently suffer from resource fragmentation, where scattered free slots prevent the allocation of large, contiguous jobs. Our Preemptive Task Reallocation algorithm actively addresses this by selectively preempting and migrating running jobs based on the expected gain/overhead. This mechanism consolidates external fragmented resources, maximizing the GPU allocation rate.
3. Our approach can determine the optimal coefficients based on job and scheduler characteristics. This means that our research demonstrates the capability to identify the optimal coefficients regardless of the specific workload presented. Deep learning is fundamentally an iterative process, this implies that the most efficient GPU cluster operation can be achieved through continuous retrospective optimization. To validate its effectiveness, we developed a scalable simulator for rapid evaluation.

Both mechanisms are essential for the overall system performance; queue adjustment prevents unfair delays, and reallocation ensures that resources are physically available for efficient scheduling.

The remainder of this paper is organized as follows: Section 2 presents several related studies. The following section explains why job-waiting queue adjustments are required. Section 4 presents the details of queue adjustment and fragmentation curation mathematically. In the subsequent section, we demonstrate how effective queue adjustment and fragmentation curation maintain a high efficiency within the GPU server pool, even with varying job characteristics. In addition, we present the performance results of applying our mechanism to job logs enhanced using actual data. Finally, we summarize this study and discuss potential directions for future development in Section 7.

Although the precise term is 'datacenter GPU', we use 'GPU' throughout this paper for simplicity, referring specifically to datacenter GPUs.

2 Related Works

2.1 Efficient LLM Scheduling by Learning to Rank

Traditional LLM serving systems use the FCFS scheduling strategy because the output generation length is typically unknown, which causes severe Head-Of-Line (HOL) blocking and reduces throughput. Efficient LLM Scheduling by Learning to Rank[9] demonstrated that accurately predicting the exact length is unnecessary; instead, knowing the relative ranks of the output lengths is sufficient to approximate the optimal SJF/SRTF scheduling. The authors proposed a novel Ranking Scheduler that uses a learning-to-rank approach implemented via a small auxiliary model (e.g., OPT-125M). The model was trained using the listwise ListMLE loss to optimize the ranking quality, which was measured using the Kendall rank correlation coefficient (Kendall's Tau), which is directly correlated with lower latency. The scheduler supports on-the-fly per-iteration scheduling and incorporates a Starvation Prevention mechanism to

ensure fairness for long requests. When integrated with state-of-the-art serving systems, the method achieved significant performance gains, including up to 2.8x lower p90 latency in chatbot serving and 6.5x higher throughput in synthetic data generation. The computational overhead of the predictor model is negligible, typically less than 2%. However, the authors also noted the following limitations: a high Kendall's Tau does not always guarantee low latency (e.g., shuffling the shortest 70% of requests maintained Tau≈0.5, but doubled the latency), and the approach was not evaluated with newer optimizations, such as request pre-filling. These factors suggest potential gaps in the generality of the method.

2.2 FGD

Despite the widespread use of GPU sharing to increase utilization, production clusters suffer from severe GPU fragmentation caused by the allocation of partial GPUs, leaving hundreds of GPUs unallocatable (a 21–42% fragmentation rate observed in traces). This fragmentation challenge requires a formulation beyond classic bin packing because GPU allocation must be contiguous, although GPUs within a node are interchangeable. The authors introduced a novel statistical fragmentation measure that quantifies the expected amount of unallocatable GPU resources given a task randomly sampled from the target workload. Based on this measure, they proposed Fragmentation Gradient Descent (FGD)[10], a heuristic that guides scheduling by selecting the node assignment that results in the minimum fragmentation increment. Evaluated on high-fidelity simulations using production traces (over 6.2k GPUs), FGD reduced unallocated GPUs by up to 49% compared to existing packing heuristics, enabling the utilization of 150–290 additional GPUs. FGD also effectively reduces stranded resources and consolidates tasks on the least-occupied nodes.

2.3 Optimus

Optimus[11] is proposed as a customized job scheduler for deep learning (DL) clusters to overcome the limitations of existing schedulers that allocate fixed resources to DL training jobs, leading to low resource efficiency and prolonged job completion times. The primary goal of Optimus is to minimize job training time and enhance resource efficiency by leveraging online resource performance models. Optimus uses online fitting to track training progress and accurately predict model convergence (steps/epochs required) and simultaneously estimates the training speed as a function of dynamically allocated resources (workers and parameter servers). Crucially, this performance modeling approach is built on runtime data and requires no prior knowledge of the internals of the ML model or hardware configuration of the cluster. Based on these predictive performance models, Optimus implements a heuristic method to dynamically allocate resources to minimize the average Job Completion Time (JCT). It also designs a task placement scheme guided by the principle of maximizing training speed using the smallest number of servers and deploying tasks evenly, thus mitigating communication overhead. In addition, Optimus identifies and resolves a significant load-imbalance issue within the parameter server architecture (e.g., MXNet) by introducing a Parameter Assignment Algorithm (PAA) that efficiently assigns model slices.

According to Kubernetes, Optimus demonstrated substantial performance gains, outperforming representative cluster schedulers (DRF and Tetris) by improving the JCT by 139%.

2.4 Themis

Themis[12] proposed a new scheduling framework that introduced finish-time fairness as a long-term fairness metric. Metric r is defined as the ratio of an app's completion time in the shared cluster (T_{sh})to its ideal completion time in a dedicated 1/N cluster (T_{id}), with the goal of minimizing the maximum r. Themis achieved this through a two-level scheduling architecture in which a central ARBITER runs round-by-round auctions based on resource leases. ML applications (apps) participate by submitting bids that reflect their performance and placement preferences through estimated r-values for different resource allocations. The auction mechanism guarantees desirable properties (strategy-proofness, Pareto efficiency, and envy-freeness) and applies Round-by-Round Filtering to maximize the sharing incentive (SI) fairness criterion. The evaluation showed that Themis improves fairness by more than 2.25X and enhances cluster efficiency by 5–250% compared with state-of-the-art schedulers.

2.5 Lucid

Lucid[8] introduced an efficient and user-friendly approach to deep learning job scheduling based on three foundational principles: ease of integration, scalability, and transparency. It avoids intrusive modifications by operating without preemption and eliminates the need for user intervention or adjustments to existing deep learning frameworks. These design choices effectively address challenges such as operational rigidity, error susceptibility, and high integration or maintenance overheads. In addition, Lucid was designed to handle complex and large-scale workloads efficiently, while providing clear and adjustable operations for system administrators. Lucid's primary focus was to reduce the average JCT for training workloads to cater to the demands of deep learning users. In addition to improving resource efficiency, it provides prompt debugging feedback. Looking ahead, Lucid aims to support additional scheduling objectives, such as ensuring fairness and meeting service-level agreements, thereby expanding its applicability to diverse workload scenarios.

2.6 Sia

Sia[7] is a scheduler specifically designed to handle resource-adaptive deep learning jobs in heterogeneous GPU clusters. It addresses the challenges of balancing the GPU type, count, and batch size assignments dynamically, which are particularly difficult because of the vast search space and diverse scaling behavior of different jobs. By leveraging an integer linear programming (ILP) formulation and pragmatically reducing the search space, Sia efficiently assigns resources while minimizing overhead. In addition, Sia employs an innovative throughput modeling approach that relies on minimal initial profiling and dynamically refines its predictions, enabling effective resource allocation without significant profiling costs. Furthermore, Sia proved effective in maintaining

fairness metrics and efficiently scaling complex hybrid parallel jobs, such as Megatron-style models[13], while maintaining robust performance across varying cluster sizes and workloads. These results highlight Sia's scalability, adaptability, and superior performance in addressing the complex challenges associated with heterogeneous resources and job variability.

2.7 Algorithms for Preemptive Co-scheduling of Kernels on GPUs

Algorithms for the preemptive coscheduling of kernels on GPUs[14] aim to minimize the number of preemptions while achieving an optimal makespan. This study demonstrated that the optimal preemptive makespan can be computed efficiently using a linear programming solution. However, determining the minimum number of preemptions among all solutions with an optimal makespan has proven to be NP-hard. These findings highlight the effectiveness of graph-based scheduling techniques in balancing computational efficiency with minimal disruption in high-demand GPU environments.

3 Motivation

Interestingly, significantly advanced deep learning technology has also led to inefficiencies in GPU server pools. Container[15]-based, reproducible, and shareable training environments, enabled by services such as Docker Hub[16] and Hugging Face[17], have greatly accelerated deep learning research worldwide. With various ML frameworks, libraries, and fundamental neural network architectures for deep learning being widely shared, unnecessary time spent on reproducibility has been minimized. Consequently, most researchers have focused on refining algorithms in a consistent environment. The GPU-dependent components of model architecture are often abstracted or hidden.

Because GPU access is abstracted, containers typically interact with GPUs at the card level. Consequently, although hardware supports time slicing to improve the GPU allocation rate, this capability cannot be flexibly utilized during model training or GPU server pool management. Moreover, as the scale of deep learning models increases, the need to use multiple GPUs simultaneously has become common. While managing a GPU server pool, an endless stream of jobs is continuously submitted, each with varying completion times and GPU counts. Consequently, fragmentation occurs within the GPU server pool, leading to inefficiency. Fragmentation occurs in the following situations.

Figs. 1 and 2 show the status of the GPU server pool and waiting job queue, respectively. A scheduler that uses the most-allocated manager can allocate tasks t_0 and t_1. However, it cannot schedule t_2 because t_2 depends on completing other jobs to free up GPU slots. This implies that tasks t_3, t_4, and t_5 can only be scheduled afterward (Fig. 3).

As illustrated in Figs. 3, despite having sufficient GPU slots to schedule t_3, t_4, and t_5, these jobs cannot be scheduled due to t_2, which is HOL blocking. This is because the previously allocated jobs cause the GPU server pool to become fragmented. Fragmentation in the GPU server pool occurs continuously during operation and is occasionally

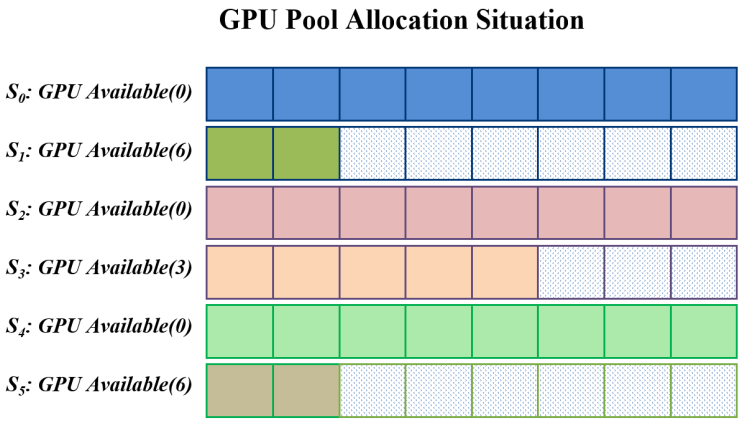


Fig. 1 GPU server pool status before newly submitted jobs are scheduled.

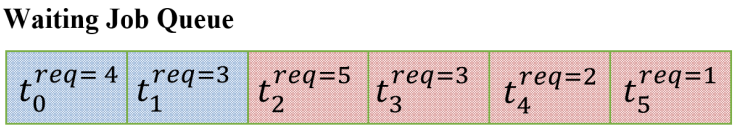


Fig. 2 Waiting job queue status before newly submitted jobs are scheduled.

resolved. The worst case occurs when there are seven GPUs available on all GPU servers and the HOL blocking requests eight GPUs.

In this case, the GPU server pool has the lowest GPU allocation rate before any job is completed. In this study, we experimentally investigated this phenomenon. Job logs were collected based on the most-allocated scheduling approach. Job logs were recorded from an actual GPU server pool. Job logs were collected over approximately three months. We analyzed the scheduling conditions based on the required GPU count. The analyses assumed a GPU server pool composed of Nvidia A30 and A100 GPUs with a single job waiting queue and no job cancellations while waiting.

The characteristics of the job log are shown in Fig. 4, which illustrates the requested number of GPUs and the training time for each job. Because of the mixture of A30 and A100 GPUs, there is an exceptionally high number of jobs requesting four GPUs, whereas the other GPU request counts follow a uniform distribution. The training time follows a Weibull minimum distribution.

We examined the frequency of HOL blocking using scheduling analysis. In particular, we observed the overall allocation rate of the GPU server pool when job groups with different GPU request counts were scheduled. For jobs requesting only one or two GPUs, scheduling can occur even at a high GPU server pool allocation rate. However, if a blockage is in front of the waiting queue, these jobs cannot be allocated until the blockage is resolved. This often results in allocation when the overall allocation rate of the server pool decreases to a certain level. Fig.5 shows that jobs requesting between

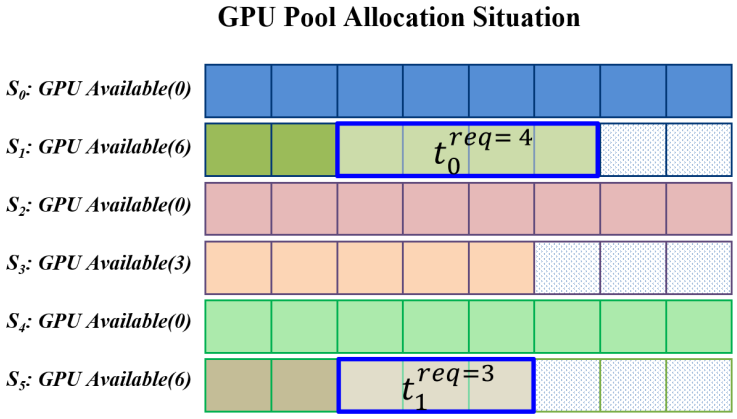


Fig. 3 GPU server pool status after some jobs are scheduled.

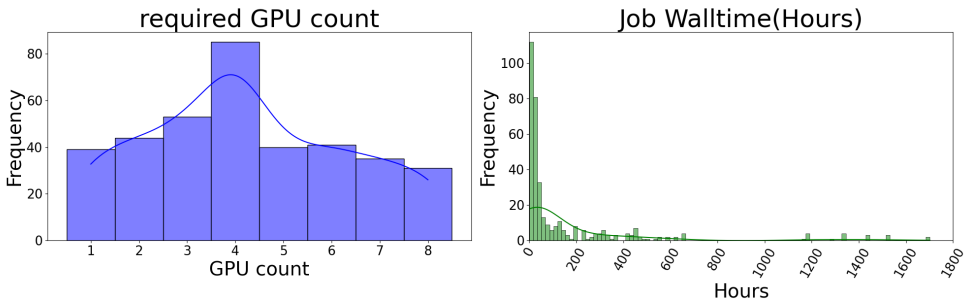


Fig. 4 GPU server pool allocation rate when job has been allocated

one and eight GPUs are allocated at similar pool allocation rates, suggesting a high frequency of blockage.

The challenge of optimally scheduling an infinite number of jobs with unknown completion and scheduling request times is NP-hard[3]. Consequently, this problem cannot be entirely resolved using scheduling algorithms alone. Consequently, this study prioritizes addressing HOL blocking rather than enhancing scheduling algorithms. Furthermore, a defragmentation technique was adopted to mitigate external fragmentation. Consequently, we introduce new mechanisms called queue adjustment and fragmentation curation.

We developed an in-house simulation for job scheduling and server adjustment to facilitate the analysis and rapid repeated development of mechanisms to resolve HOL blocking. It parses job logs and responds to scheduling using different methods. New scheduling methodologies can be added without affecting existing mechanisms. This implies that the queue and fragmentation adjustments operate independently of the job-scheduling method.

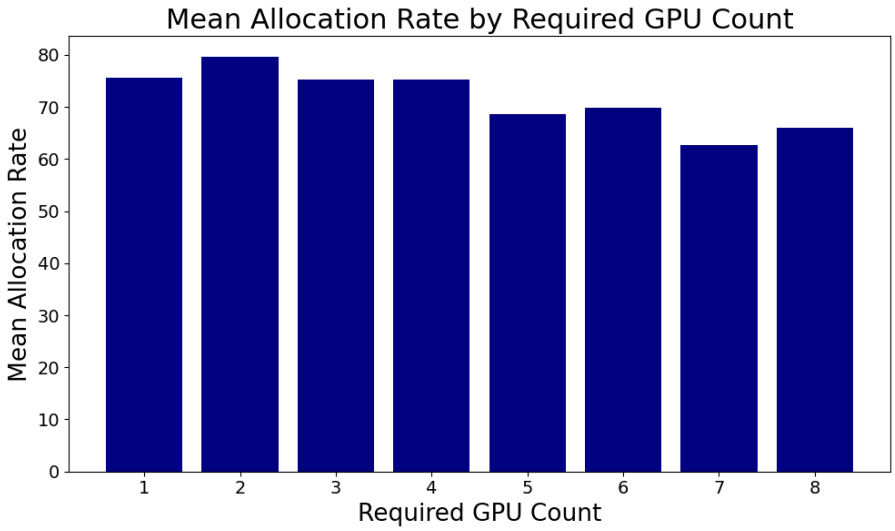


Fig. 5 Actual job log characteristics.

4 Resolving HOL blocking in Job Queue and GPU Fragmentation

We propose two approaches to address HOL blocking in the job-waiting queue and fragmentation of the GPU server pool. First, we introduce a starvation-free algorithm to reduce HOL blocking. Second, we develop a preemptive task reallocation algorithm to minimize fragmentation in the GPU server pool. Both algorithms aim to optimize the usage of the GPU server pool. Each algorithm functions independently and has its own set of trade-offs. These trade-offs are determined using the coefficients applied to each algorithm.

4.1 Starvation-free Algorithm

The advanced queue and fragmentation management proposed in this study aims to ensure a high allocation rate for the GPU server pool, regardless of the type or method of scheduler used. The starvation-free and preemptive task reallocation algorithms function before scheduler execution. Together, these two algorithms optimize the job-waiting queue and server states, enabling optimal scheduling performance independent of the scheduler type or mechanism.

HOL blocking frequently occurs during GPU server pool operations and is eventually resolved; however, this resolution can be time-consuming. This delay negatively affects the overall allocation rate. Scheduling to prevent HOL blocking is an NP-hard problem, and it is nearly impossible to predict when HOL blocking occurs. This is because of the unpredictable nature of the types of jobs that request GPUs and the resources required in the real world. Therefore, scheduling to prevent HOL blocking is an indirect approach. Starvation-free algorithms address blockage issues by directly

modifying the priorities of jobs in the waiting queue. We also consider the trade-offs that arise from these modifications using specific coefficients. Although adjusting the queue can enable jobs blocked by a blockage to be scheduled, there is a trade-off in that, in the worst-case scenario, jobs associated with the blockage may never be allocated. This algorithm was designed to prevent starvation in top-priority jobs. Arbitrarily eliminating HOL blocking or adjusting the priorities can lead to fairness issues.

Let P^* represent the adjusted priority value of tasks t_0^*, t_1^*, t_2^*, ..., t_n^*. P^* is calculated and reassigned before scheduling is initiated. The scheduler uses modified job priorities. The adjusted job priority P^* is defined as:

$$P^* = \begin{cases} P + \alpha A \times R, & \text{if } \beta > AR \\ P, & \text{otherwise} \end{cases} \tag{1}$$

The first term P represents the original priority of tasks t_0, t_1, ..., t_n, where n is three times the number of servers. Specifically, the priorities are defined as follows: $p_0 = 1$, $p_1 = 0.5$, and $p_2 = 0.25$, with each subsequent priority decreasing by half; a higher value indicates a greater priority than a lower one. The second term consists of the variables $\alpha A \times R$ which represents the starvation index. Here, α is the coefficient that dictates the rate at which the starvation index gains importance. For example, to ensure that the priority of the second job in the waiting queue increases within six hours, assuming that R is set to one, α should be set to at least 0.013889. In numerical terms, when P_1 is 0.5, the calculation $0.013889 \times 36 \times 1 = 0.500004$ results in an updated priority of 1.000004, surpassing the P_0 value of 1, thereby shifting the priority. In this context, A refers to the age of the task, which increases by one every ten minutes, and R is the resource suitability index. A is reset when a job is scheduled. The value of R is one when the number of required accelerators exactly matches the available number. However, it decreases by 0.1 for each additional accelerator. Finally, β is the minimum allocation rate that determines whether the starvation-free algorithm should be activated. The starvation-free algorithm is triggered if the allocation rate AR falls below this threshold. Algorithm 1 shows the implementation of a starvation-free algorithm using (1).

4.2 Preemptive task reallocation Algorithm

The preemptive nature of accelerator-based learning tasks implies that they can be paused or resumed from the paused points using checkpoints. This characteristic can be utilized to resolve fragmentation in the server pool. Resolving fragmentation is not about adjusting to the current queue but rather about consolidating preemptive tasks in one place to increase the number of servers without job allocation. Preemptive jobs are moved to maximize the number of servers with no assigned jobs. Dynamic programming is used to determine the optimal reallocation. There are two main trade-offs when using dynamic programming for job reallocation. The first is the execution time of the dynamic programming algorithm. The goal is not to find the optimal placement for all preemptive jobs but to maximize the number of servers with no assigned jobs. Given the small scale of the server pool, redundant optimal placements can be found quickly; however, the total number of possible configurations for all preemptive jobs

Algorithm 1 Starvation-free Algorithm

Input:

Server count : $\boldsymbol{n}$, Wait queue length : $\boldsymbol{l_{wq}}$
Servers :$\boldsymbol{S_i} \in \{S_0, S_1, \ldots, S_{n-1}\}$,
Accelerator count in S_i: $\boldsymbol{M_i} \in \{M_0, M_1, \ldots, M_{n-1}\}$,
Resource suitability index : $\boldsymbol{R_i} = 0, \forall i \in \{1, 2, \ldots, 8\}$,
Wait Queue : $\boldsymbol{WQ_i} \in \{WQ_0, WQ_1, \ldots, WQ_{l-1}\}$,
Task in WQ : $\boldsymbol{t_i} \in \{t_0, t_1, \ldots, t_{l-1}\}$,
Accelerator request of t_i : $\boldsymbol{a_{t_i}}$
Wait Age : $\boldsymbol{A_i} \in \{A_0, A_1, \ldots, A_{m-1}\}$

Procedure: Updated wait queues

if *latest allocation* $\geq \beta$ **then**
  return
end if

$R \leftarrow 0$

for $i \leftarrow 0$ to $n-1$ **do**
  for $j \leftarrow 1$ to M_i **do**
    if $a_{S_i} > j$ **then**
      $R_j \leftarrow \max\{R_j, 1 - (a_{S_i} - j) \times 0.1\}$
    end if
  end for
end for

for $i \leftarrow 0$ to $l_{wq} - 1$ **do**
  $P_i \leftarrow 1/2^i$
  $P_i^* \leftarrow P_i + \alpha A_i \times R_{a_{t_i}}$
end for

$(P^*_{i_{\max}}, i_{\max}) \leftarrow \max(P^*)$
$P_0 \leftarrow P^*_{i_{\max}}$

for $j \leftarrow 1$ to $i_{\max}$ **do**
  $P_j \leftarrow P^*_{j-1}$
end for

is extremely large. Consequently, if all configurations are considered, the time cost of dynamic programming may outweigh the benefits of defragmentation. The second trade-off is the time required for reallocation. Using checkpoints for reallocation does not operate at the OS kernel level; instead, it involves stopping the job, performing reallocation, and resuming execution because downtime is required for this operation.

It is essential to find an appropriate balance between total downtime and the benefits gained from reallocation.

$$dp(i, a_0, a_1, \ldots, a_{n-1}, \text{calls}) = F_i \tag{2}$$

Equation (2) represents the dynamic programming, dp stands for dynamic programming, for the maximum number of servers F_i with exactly the total accelerator count in the server left after the reallocation task i, where $a_0, a_1, \ldots, a_{n-1}$ are the available hardware accelerators on each server, and calls denotes the number of recursive calls. Each server S_j begins with tasks already allocated, implying that its available hardware accelerators are reduced based on the tasks already placed on that server. Equation (3) shows the initial status of dp.

$$dp(0, a_0, a_1, \ldots, a_{n-1}, 0) = F_0 \tag{3}$$

where a_j is the current number of available hardware accelerators on server j. The maximum capacity of each server is $M_j = 4$ or $M_j = 8$ depending on the server type. The dp process begins only if the waiting queue length is satisfied; that is, the wait queue (WQ) length $\geq \omega$. t_r is a running task on the server that performs preemptive operations. r denotes the recursive call depth.

$$\begin{aligned} dp(i, a'_0, a'_1, \ldots, a'_{n-1}, \text{calls}) = \\ max_{i \neq S_i} \big(dp(i-1, a_0, a_1, \ldots, a_{n-1}, \text{calls}-1), \\ dp(i, a_0, a_1, \ldots, a_{n-1}, \text{calls}) + \sum_{j=0}^{n-1} f(a_j, t_r, M_j) \big) \\ , where\ WQ \geq \omega. \end{aligned} \tag{4}$$

where $i \neq S_i$ indicates that task t_i does not need to be reassigned to the original server S_i. Once the calls reach δ, recursion stops. When task t_i is reassigned from server S_i, the resources used by the task are released; that is, $a'_{S_i} = a_{S_i} + a_{t_i}$. After reassigning task t_i to server j (with $j \neq S_i$), the available hardware accelerators on server j are updated as $a'_j = a_j - a_{t_i}$. In addition, $a'_j \geq 0$ must hold, implying that the server cannot be overcommitted.

$$f(a_j, t_r, M_j) = \begin{cases} 1 & \text{if } a'_j = M_j, \\ 0 & \text{otherwise.} \end{cases} \tag{5}$$

Equation (5) checks whether, after task reassignment, server j has exactly the maximum number of remaining hardware accelerators (either four or eight, depending on M_j).

$$dp(m, a_0, a_1, \ldots, a_{n-1}, \text{calls}) = F_m, \text{if } F_m > F_0. \tag{6}$$

Equation (6) presents the final state that reallocates preemptive tasks to the most reasonable server as a result of dynamic programming. Therefore, Algorithm 2 represents the detailed sequence of dynamic programming.

The queue adjustment and fragmentation curation proposed in this study aim to maintain a high allocation rate for the GPU server pool, regardless of the scheduler type or method. The starvation-free and preemptive task reallocation algorithms function before scheduler execution. Together, these two algorithms optimize the job-waiting queue and server states, enabling an optimal scheduling performance independent of the scheduler type or mechanism.

The two algorithms operate independently of the scheduler and are executed before the scheduler, which operates at regular intervals. They rely solely on the job-waiting queue and current job allocation status within the server pool to determine their actions. Because starvation-free and preemptive task reallocation algorithms are not interdependent, they can function separately, and users may opt to run only one of the algorithms if desired. This study directly manages the job-waiting queue and jobs allocated to the servers, making it incompatible with schedulers that also perform job scheduling for these elements. Therefore, this study did not target schedulers that rely on cumulative metrics or contexts.

5 Optimization of Coefficient α, β, ω, and δ

The starvation-free and preemptive task reallocation algorithms utilize the coefficients α, β, ω, and δ to control the trade-offs. This section presents an analyses on the impact of the four coefficients on performance improvement and derives an objective function to determine the optimal values for each coefficient. This approach enables the calculation of coefficients tailored to various job behaviors.

5.1 Correlation between performance and coefficients

All four coefficients must be calculated and applied to ensure accurate operation. The impact of each coefficient was measured using job logs to determine the optimal values. The coefficient ranges are listed in Table. 1.

Table 1 Coefficient Range

Coefficient	α	β	ω	δ
Range	0.01 to 1.99	80% to 95%	10 to 100	100k to 500k
Step	0.01	5%	10	100k

First, we analyze the impact of each coefficient on the completion time of all jobs in the log using SHAP[18] values. The results of the SHAP analysis are shown in Fig. 6. The four-coefficient SHAP value analysis yielded 40,086 possible combinations.

Fig. 6 analyzes the impact of each coefficient on overall job completion time, or wall time. The coefficient β has a strong negative correlation, where higher values

Algorithm 2 Preemptive task reallocation Algorithm

Input:

Server count : $\boldsymbol{n}$, Wait queue length : $\boldsymbol{l_{wq}}$,
Memorization cache : $\boldsymbol{mc}$, Full empty server count : $\boldsymbol{F}$
Static call count : $\boldsymbol{calls} \leftarrow 0$, Recursive call depth : $\boldsymbol{r}$,
Servers :$\boldsymbol{S_i} \in \{S_0, S_1, \ldots, S_{n-1}\}$,
Shadow Servers :$\boldsymbol{S_i^{'}} \in \{S_0^{'}, S_1^{'}, \ldots, S_{n-1}^{'}\} \leftarrow S$,
Accelerator count in S_i: $\boldsymbol{M_i} \in \{M_0, M_1, \ldots, M_{n-1}\}$,
Available accelerator count in $S_i^{'}$: $\boldsymbol{a_i}$,
Preemptive task : $\boldsymbol{t_j}, \in \{t_0, t_1, \ldots, t_{j-1}\}$
Wait Queue : $\boldsymbol{wq_i} \in \{wq_0, wq_1, \ldots, wq_{l-1}\}$,

Output:

Max Full empty Server count : $\boldsymbol{F}$, Final status servers $\boldsymbol{S^{'}}$

Procedure(DP for reallocation): $dp(r, F, calls)$

if $\boldsymbol{l_{wq}} < \omega$ **then**
  return -1
end if

if $calls > \delta$ **then**
  return F
end if

$calls \leftarrow calls + 1$

if $MC[allocation\_state]$ existed **then**
  return $MC[allocation\_state]$
end if

if $calls == 0$ **then**
  Clear MC
end if

if $r == l_{wq}$ **then**
  return F
end if

for $i \leftarrow 0$ to $n-1$ **do**
  $F \leftarrow \max(F, \sum_{j=0}^{n-1} f(a_j, t_r, M_j))$
  $MC[allocation\_state] \leftarrow F$
  $F \leftarrow dp(r+1, F, calls)$
end for

$F \leftarrow dp(r+1, F, calls)$

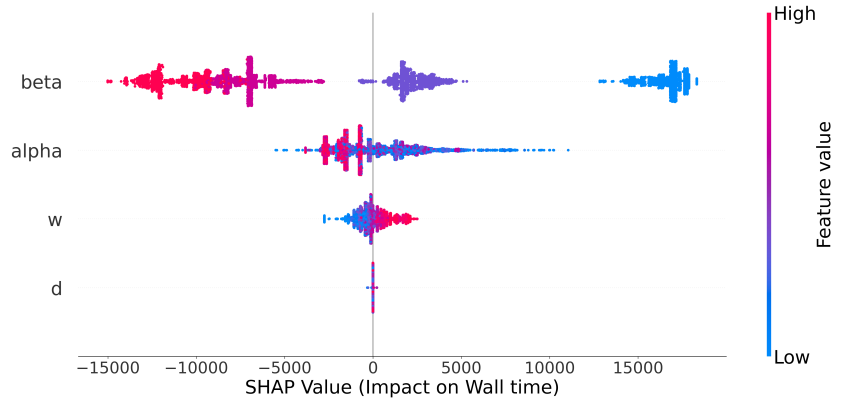


Fig. 6 SHAP value analysis of coefficients.

of β correspond to reduced wall time. The coefficient α also negatively correlates with the wall time but exerts a weaker influence than β. The coefficient δ shows no significant positive or negative correlation with wall time. The coefficient ω exhibits a weak positive correlation, indicating that lower values of ω slightly reduce the wall time, although its influence is less than that of α.

5.2 Trade-off analysis and definition of related metrics

To optimize the four coefficients, it is essential to calculate the trade-offs. The coefficients α and β help eliminate HOL blocking but may also cause blocked jobs to remain unscheduled indefinitely, leading to starvation or extended delays. To prevent this, the appropriate coefficients must be set. Thus, the cumulative waiting time for each job is grouped by the number of requested accelerators and calculated as a trade-off, which we define as the accumulated average distance (AAD).

$$AAD = \sum_{k=1}^{8} \frac{(WT'_k - WT_k)}{2^{8-k}} \tag{7}$$

Equation (7) measures the weighted average difference between the waiting times before and after applying α and β. Let the original sum of the waiting times of the jobs grouped by the requested hardware accelerators count be denoted by $WT_1, \ldots, WT_8$.

After applying α and β, the updated waiting times are WT'_1, WT'_2, $\ldots$, WT'_8. The distance between waiting times is calculated as the average accumulated distance. In addition, the distance is halved each time the number of requested accelerators decreases.

The accumulated maximum distance is the value selected from the waiting time differences. w_t represents the set of waiting times for jobs based on the requested hardware-accelerator count. Therefore, w'_t represents the set of maximum waiting times for each requested hardware accelerator count.

Equation (8) shows the accumulated maximum distance (AMD).

$$AMD = \sum_{k=1}^{8} \max\left(\frac{w'_t - w_t}{2^{8-k}}\right) \tag{8}$$

The computation of these metrics ensures that both the average and maximum deviations in task waiting times are considered, enforcing fairness in scheduling and preventing any significant performance deterioration owing to starvation or excessive waiting.

The preemptive task reallocation algorithm takes longer when dynamic programming is performed, indicating the additional time required compared to not using this algorithm. The trade-off of the preemptive task reallocation algorithm is measured using the algorithm execution time (AET).

5.3 Objective functions

The coefficients α and β are adjusted to minimize the wall time, AAD, and AMD. To determine the optimal values of these two coefficients, a grid search over a range of α and β values is performed. The wall time, AAD, and AMD are computed for each combination of α and β. All three metrics are normalized for the min–max scaler to ensure fair comparison across the metrics. An objective function is defined to aggregate the normalized values of the three. The objective function of the starvation-free algorithm is given by (9). The objective function has a simple formation. Hence, the minimum value is determined using the min–max scalar. δ and ω are calculated using the min–max scalar.

$$Obj_i^{sfa} = walltime_i^{norm} + AAD_i^{norm} + AMD_i^{norm} \tag{9}$$

The objective function for δ and ω aims to minimize both the wall time and overhead introduced by preemptive task reallocation. This is measured indirectly using the dynamic programming execution time (ms) (DPE). In addition, this algorithm aims to maximize preemption gain. The preemption gain is the measured gap between the original and algorithm-applied wall times. It marks as PMG. Equation (10) represents the objective function for δ and ω.

$$Obj_i^{ptr} = walltime_i^{norm} + DPE_i^{norm} + PMG_i^{norm} \tag{10}$$

6 Experimental Evaluation

Queue adjustment and fragmentation curation are simple, flexible, and powerful mechanisms. To the best of our knowledge, no existing method directly addresses HOL blocking in the job-waiting queue and fragmentation within the servers. To validate the flexibility and performance of the queue adjustment and fragmentation curation, we identified the optimized coefficients for various job distributions and evaluated the improvements in scheduling performance using these coefficients. To identify the optimal coefficients for various job distributions, we simulated the accelerator requests and

job execution times using eight distributions: normal, exponential, log-normal, gamma, beta, Weibull minimum, uniform, and Poisson. All generated job logs contained 1,000 jobs. The ranges of the coefficients in the search space are listed in Table. 1 However, only the range for ω was modified to span from 20 to 200 in increments of 20.

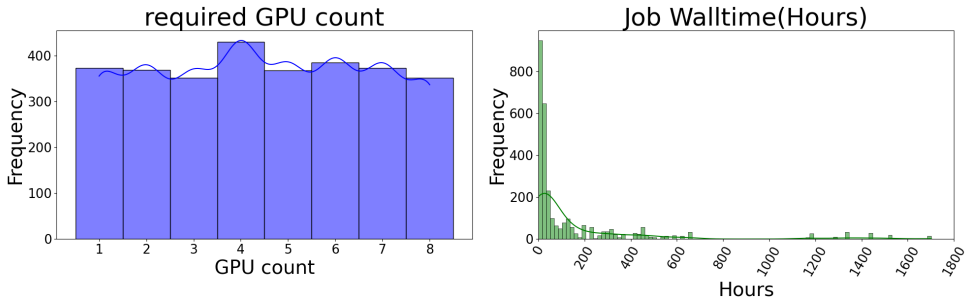


Fig. 7 Augmented job log characteristics.

To measure the performance improvement, we used two types of job logs: actual logs from Section III and augmented job logs based on the actual logs. These logs contained 368 and 3,001 job entries, respectively. The evaluation used two job logs to compare the results before and after applying queue adjustment and fragmentation curation. The four coefficients used were identical to those in the search space above. The characteristics of the augmented job logs are presented in Fig. 7. It includes the requested number of GPUs and wall time for each job.

A simulator was employed to determine the optimal coefficients and compare the performance of the mechanisms. The simulations were executed on a Windows 11 system with an Intel Core Ultra 7 155H 3.80 GHz processor and 32GB of RAM, which was developed in C++ using Visual Studio Community 2022.

6.1 Optimal value of coefficient α, β, ω, and δ

The simulation results for the eight types of distribution job logs represent the optimal value of each coefficient. Fig. 8. presents the characteristics of each job log. Table 2. shows the optimal values of the coefficients α, β, ω, and δ for each distribution and the time reduction achieved by the algorithm. Additionally, Table 2 shows that the optimal coefficients vary according to job-log characteristics. This suggests that rather than using fixed coefficients, it is beneficial to adjust them according to the specific characteristics of the workload.

The results demonstrate that queue adjustment and fragmentation curation operate effectively across various types of workloads. Furthermore, it highlights the importance of fine-tuning the coefficients to match the specific characteristics of each workload.

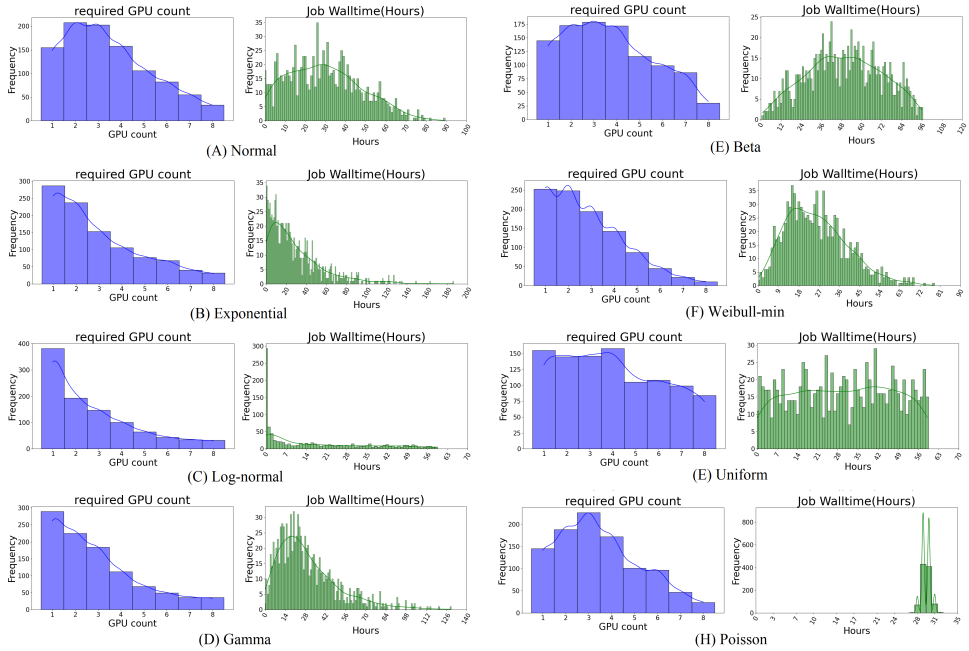


Fig. 8 Eight types of job distribution characteristics.

Table 2 Optimal Value of Coefficients

Distribution methodology	α	β	ω	δ
Normal	0.53	85%	60	200k
Exponential	0.44	90%	200	400k
Log-normal	1.63	95%	20	100k
Gamma	0.22	95%	80	400k
Beta	0.26	90%	180	100k
Weibull minimum	0.55	90%	120	400k
Uniform	0.5	95%	200	400k
Poisson	0.22	90%	100	300k

6.2 Evaluation of scheduling improvement

The previous optimal search method for coefficients allowed us to define the coefficients for both the actual and augmented job logs. The coefficients are listed in Table 3 and Figs. 9(A) and 9(B) illustrate the effects of the optimal coefficients on wall time, the trade-off metric, the allocation rate, and the utilization rate when simulating using both job logs. The X-axis represents the time at which one unit corresponds to 10 min. GPU server pool allocation and total utilization were also obtained. In Figs. 9(C) and 9(D), both the allocation rate and utilization of the actual job log shift to the right, indicating a higher allocation rate and utilization.

Table 3 Optimal Coefficient Values for Job Log

Coefficient	Actual job log	Augmented job log
α	0.03	0.72
β	90 %	90%
ω	10	200
δ	200k	300k

Table 4 lists the wall times for each algorithm combination. The simulation results for the actual and augmented job logs exhibited different behaviors. Before applying the queue adjustment and fragmentation curation wall, the times of the actual job log and augmented job were 165 days, 2 h, and 1 min, and 1,547 days, 6 h, and 1 min, respectively. After applying the algorithm, the wall times of the two job logs were reduced by approximately 13 and 80 days, respectively. The actual job log achieved a performance gain of 7.8 % during the wall time. A performance gain of 14.98 % was achieved for the archived augmented job logs. The results are presented in Fig. 9(A), Fig. 9(B), and Table 4. An augmented job log was generated based on the actual job log. The first part of the augmented job log consists of 368 jobs identical to those in the actual job log. Subsequently, augmentation was performed to reflect the behavior of the actual job logs. As a result of this augmentation, the dominance of the job group requiring four GPUs diminished. Consequently, the augmented job log was approximately 8.1 times larger than the actual job log. A large number of logs indicates that fragmentation in the GPU server pool intensifies as the workload accumulates over time.

Table 4 Walltime of Simulation Results

Applied Algorithm	Actual job log	Augmented job log
Normal	165d 02h 01m	1,547d 06h 01m
All algorithms	152d 02h 20m	1,464d 08h 07m
Starvation-free algorithm only	151d 01h 38m	1,464d 06h 22m
Preemptive task reallocation algorithm only	165d 10h 01m	1,545d 17h 03m

Table 4 shows the differences in wall time when both algorithms are applied together compared to when each algorithm is applied individually. The reduction in the overall job processing time when applying both algorithms or the starvation-free algorithm alone is consistent for both actual and augmented job logs. However, when only the preemptive task reallocation algorithm is applied, the wall time for the actual job log increased compared to when no algorithm was applied. This is because, with fewer jobs, defragmentation can exacerbate HOL blocking.

However, from the perspective of the allocation rate, applying each algorithm individually to the actual job log achieves a higher allocation rate than applying no algorithm. This is shown in Fig. 10(A). Similarly, for the augmented job log, both

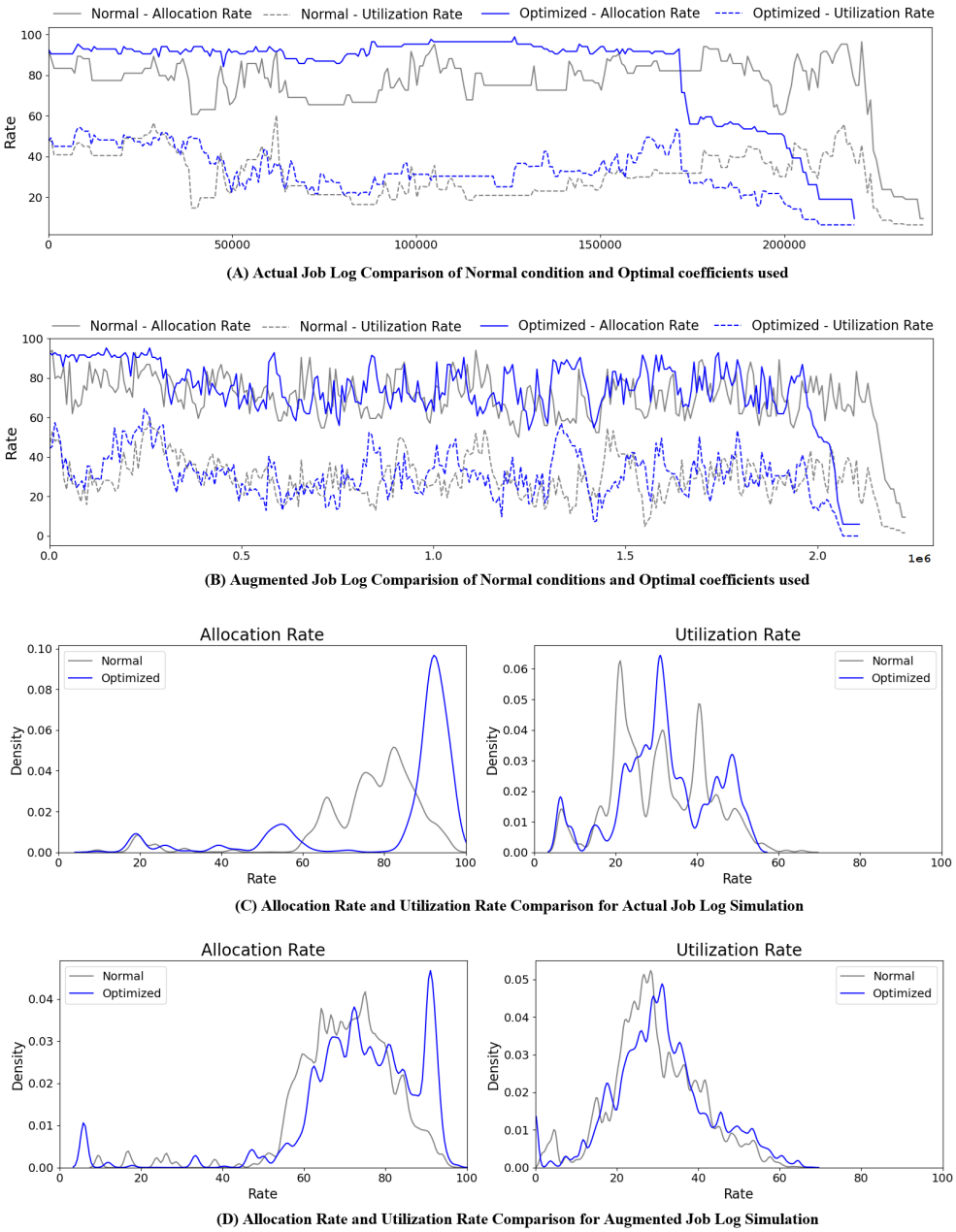
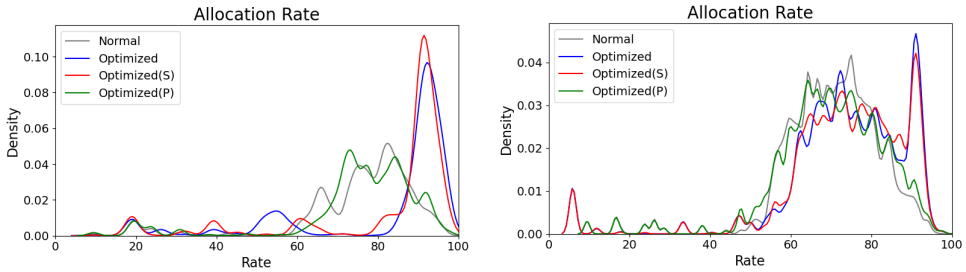


Fig. 9 Simulation results comparison for two job log.

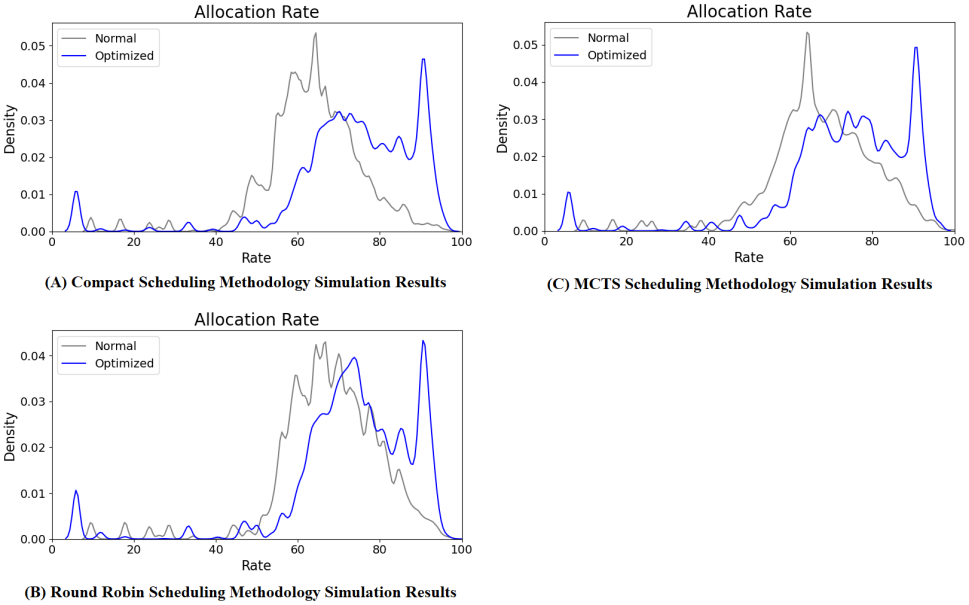
the individual and combined algorithms achieve higher allocation rates, as shown in Fig. 10(B). In Fig. 10(A), the application of the preemptive task reallocation algorithm, denoted as Optimized(P), shows a slightly higher allocation rate, indicated by



(A) Actual Job Log Simulation results **(B) Agumented Job Log Simulation results**

Fig. 10 Allocation rate comparison of algorithm combination.

a slight shift to the right compared to the normal case. When only the starvation-free algorithm or both algorithms were applied, the allocation rate increased significantly, denoted as Optimized(S). In addition, when both algorithms were applied, a marginal shift to the right was observed, which was denoted as Optimized. The effectiveness of the algorithms is even more evident in the augmented job log simulation results shown in Fig. 10(B), where higher allocation rates are observed, even when only the starvation-free algorithm is applied.



(A) Compact Scheduling Methodology Simulation Results **(C) MCTS Scheduling Methodology Simulation Results**

(B) Round Robin Scheduling Methodology Simulation Results

Fig. 11 Allocation rate comparison of various scheduling methodologies.

Queue adjustment and fragmentation curation handle the job-waiting queue and GPU server pool directly. Therefore, this is an agonistic mechanism against the scheduler methodologies. Previous evaluations used the most-allocated scheduler. Table 5

lists the queue adjustment and fragmentation curation archiving performance gains for the other three schedulers. An augmented job log was used; therefore, the same coefficients used in the previous simulations used an augmented job log.

Table 5 Other Scheduler Methodology Applying Results

	Compact	**Round robin**	**MCTS**
Normal	1,700d 11h 01m	1,588d 18h 01m	1,605d 08h 01m
Optimal	1,462d 20h 30m	1,467d 08h 27m	1,462d 11h 31m
Gain[a]	238d (14%)	121d (7.6%)	143d (8.9%)

[a]Days Count Only

Compact and round robin scheduling methodologies are simple. The MCTS-Monte Carlo Tree Search[19] is a search method based on randomness. Recently, several studies have introduced MCTS for resource scheduling, such as Hofmann1 C[20]. and Kung H[21]. In the simulations using MCTS in this study, the most allocated method was employed. A comparison of allocation rates for each scheduling methodology is presented in Fig. 11. As shown in the figure, the optimized coefficients consistently shift the allocation rate distribution toward higher utilization across all three scheduling methodologies—Compact, Round Robin, and MCTS—demonstrating the scheduler-agnostic nature of the queue adjustment and fragmentation curation mechanisms. Table 5 quantitatively confirms this improvement. The wall time was reduced by approximately 14%, 7.6%, and 8.9% for the Compact, Round Robin, and MCTS schedulers, respectively. These results indicate that the proposed optimization not only improves GPU allocation efficiency but also provides stable performance gains, regardless of the underlying scheduling strategy.

7 Conclusions and Future Works

This study proposes a queue adjustment and fragmentation curation mechanism to maintain a high allocation rate in GPU server pools. Unlike conventional approaches used by general GPU schedulers, this mechanism reduces the occurrence of HOL blocking, which degrades the scheduling efficiency and eliminates fragmentation in GPU server pools, thereby enhancing the efficiency of various schedulers. Comprising the starvation-free algorithm, which addresses HOL blocking, and the preemptive task reallocation algorithm for defragmentation, queue adjustment, and fragmentation curation the wall time was reduced by 7.8% for the actual job log and 4.9% for the augmented job log.

In addition, this study precisely defines the trade-offs introduced by these two algorithms and proposes methods to measure them. To balance the trade-offs and performance gains, the coefficients α, β, ω, and δ were defined and calculated. The coefficients were derived not only for the job logs used in the experiments but also for various probability distributions, demonstrating the versatility of the proposed mechanism. Queue adjustment and fragmentation curation mechanisms demonstrated

wall time reductions of 4.9%, 14%, 7.6%, and 8.9% when applied alongside scheduling methods, such as most-allocated, compact, round robin, and MCTS, respectively. This effectiveness is attributed to the scheduler-neutral operation of the mechanism. This mechanism remains unaffected by the specific scheduling method used, which directly adjusts the job queue and server states.

However, identifying the optimal coefficients relies on a posterior approach involving numerous iterations, similar to hyperparameter tuning in deep learning. Although this method is suitable for research purposes, it has limitations in real-world scenarios, where the coefficients must be precomputed to align with the workload characteristics. To address this limitation, future studies should focus on dynamically calculating the coefficients in real time to determine and apply the most suitable coefficients in the current context. Given that these algorithms enable the precise calculation of costs and rewards, the application of reinforcement learning in real-time scenarios is a promising direction for improvement.

References

[1] Athlur, S., Saran, N., Sivathanu, M., Ramjee, R., Kwatra, N.: Varuna: scalable, low-cost training of massive deep learning models. In: Proceedings of the Seventeenth European Conference on Computer Systems, pp. 472–487 (2022)

[2] Weng, Q., Yang, L., Yu, Y., Wang, W., Tang, X., Yang, G., Zhang, L.: Beware of fragmentation: Scheduling GPU-Sharing workloads with fragmentation gradient descent. In: 2023 USENIX Annual Technical Conference (USENIX ATC 23), pp. 995–1008. USENIX Association, Boston, MA (2023). https://www.usenix.org/conference/atc23/presentation/weng

[3] Robson, J.M.: Storage allocation is np-hard. Information Processing Letters **11**(3), 119–125 (1980) https://doi.org/10.1016/0020-0190(80)90124-6

[4] Chen, Y., Guan, Z., Shao, X.: A comparative analysis of job scheduling algorithm. In: MSIE 2011, pp. 1091–1095 (2011). IEEE

[5] Hochba, D.S.: Approximation algorithms for np-hard problems. ACM Sigact News **28**(2), 40–52 (1997)

[6] Ye, Z., Gao, W., Hu, Q., Sun, P., Wang, X., Luo, Y., Zhang, T., Wen, Y.: Deep learning workload scheduling in gpu datacenters: A survey. ACM Computing Surveys **56**(6), 1–38 (2024)

[7] Jayaram Subramanya, S., Arfeen, D., Lin, S., Qiao, A., Jia, Z., Ganger, G.R.: Sia: Heterogeneity-aware, goodput-optimized ml-cluster scheduling. In: Proceedings of the 29th Symposium on Operating Systems Principles, pp. 642–657 (2023)

[8] Hu, Q., Zhang, M., Sun, P., Wen, Y., Zhang, T.: Lucid: A non-intrusive, scalable and interpretable scheduler for deep learning training jobs. In: Proceedings of the

28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, pp. 457–472 (2023)

[9] Fu, Y., Zhu, S., Su, R., Qiao, A., Stoica, I., Zhang, H.: Efficient llm scheduling by learning to rank. Advances in Neural Information Processing Systems **37**, 59006–59029 (2024)

[10] Weng, Q., Yang, L., Yu, Y., Wang, W., Tang, X., Yang, G., Zhang, L.: Beware of fragmentation: Scheduling {GPU-Sharing} workloads with fragmentation gradient descent. In: 2023 USENIX Annual Technical Conference (USENIX ATC 23), pp. 995–1008 (2023)

[11] Peng, Y., Bao, Y., Chen, Y., Wu, C., Guo, C.: Optimus: an efficient dynamic resource scheduler for deep learning clusters. In: Proceedings of the Thirteenth EuroSys Conference, pp. 1–14 (2018)

[12] Mahajan, K., Balasubramanian, A., Singhvi, A., Venkataraman, S., Akella, A., Phanishayee, A., Chawla, S.: Themis: Fair and efficient {GPU} cluster scheduling. In: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), pp. 289–304 (2020)

[13] Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., Catanzaro, B.: Megatron-lm: Training multi-billion parameter language models using model parallelism. arXiv preprint arXiv:1909.08053 (2019)

[14] Eyraud-Dubois, L., Bentes, C.: Algorithms for preemptive co-scheduling of kernels on gpus. In: 2020 IEEE 27th International Conference on High Performance Computing, Data, and Analytics (HiPC), pp. 192–201 (2020). IEEE

[15] Linux Container. http://linuxcontainers.org. Accessed: Oct. 11, 2025

[16] Docker Hub. https://hub.docker.com/. Accessed: Oct. 11, 2025

[17] Hugging Face. https://huggingface.co/. Accessed: Oct. 11, 2025

[18] Lundberg, S.M., Lee, S.-I.: A unified approach to interpreting model predictions. Advances in neural information processing systems **30** (2017)

[19] Coulom, R.: Efficient selectivity and backup operators in monte-carlo tree search. In: International Conference on Computers and Games, pp. 72–83 (2006). Springer

[20] Hofmann, C., Liu, X., May, M., Lanza, G.: Hybrid monte carlo tree search based multi-objective scheduling. Production Engineering **17**(1), 133–144 (2023)

[21] Kung, H.-L., Yang, S.-J., Huang, K.-C.: An improved monte carlo tree search approach to workflow scheduling. Connection science **34**(1), 1221–1251 (2022)